DBA5103

Operations Research and Analytics

Assignment 3

Semester I, Academic Year 2026/2027

Reagan (CHAN HIU LOK)
MSBA | NUS
Student No: A0319086X

1. Process Flexibility

For a given design, that is, a set of arcs A linking plants to models, and a realised demand vector D, the maximum weekly sales solve a network flow problem in which x_ij is the amount of model j produced at plant i:

$$f(A, D) = \max \sum_{(i,j) \in A} x_{ij}$$

$$\text{s.t.} \quad \sum_{j\,:\,(i,j) \in A} x_{ij} \le 100, \qquad \sum_{i\,:\,(i,j) \in A} x_{ij} \le D_j, \qquad x_{ij} \ge 0.$$

Demand is random, so each design is evaluated by simulation. Draw demand samples with D_j = max{X_j, 0}, where the X_j are independent N(100, 30^2), solve the network flow problem for every sample, and average the optimal sales. The open chain adds the five arcs from plant i to model i + 1 to the dedicated design; the long chain adds one further arc, from plant 6 back to model 1, which closes the chain into a single cycle. The same 1,000 demand samples (and, as a robustness check, 10,000 samples) are used for every design so that the comparison is not distorted by sampling noise.

Design	Arcs	Avg. sales (1,000 samples)	Std. error	Avg. sales (10,000 samples)	Share of full-flexibility gain
Dedicated	6	527.2	1.32	527.7	0%
Open chain	11	556.4	1.37	556.0	67%
Long chain	12	570.1	1.33	570.1	100%
Full flexible	36	570.2	1.34	570.2	100%

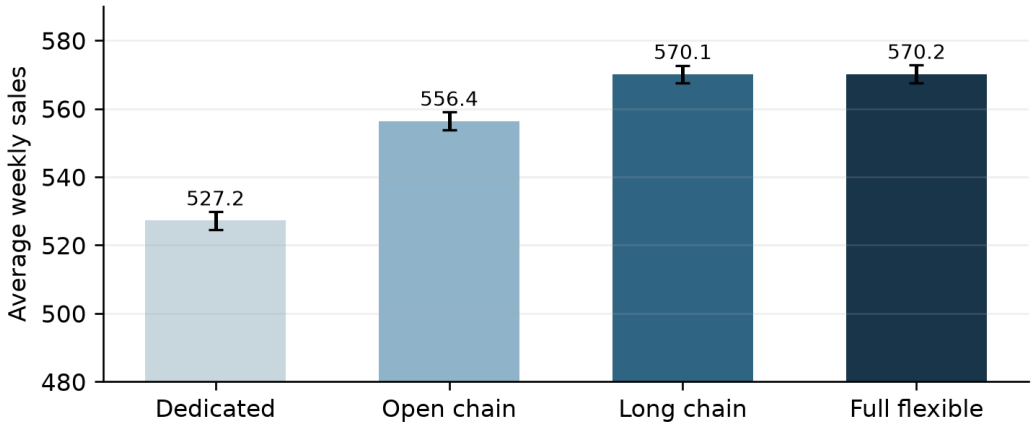


Figure 1. Average weekly sales by design (1,000 samples, 95% intervals)

Answer: The open chain sells about 556 units per week on average and the long chain about 570, against about 527 for the dedicated design and 570 for full flexibility.

The open chain recovers roughly 67% of the gain from full flexibility using 11 arcs instead of 36. The long chain, with just one more arc, recovers essentially all of it (100%) and is statistically indistinguishable from the fully flexible system. The reason is the end of the open chain: model 1 can only be made at plant 1 and plant 6 can only make model 6, so a demand surge for model 1 or spare capacity at plant 6 cannot be shared with the rest of the system. Closing the chain gives every plant and every model two links, and excess demand anywhere can be passed along the cycle in either direction to a plant with spare capacity. A small amount of well-placed flexibility therefore delivers almost all of the benefit of full flexibility.

2. Multi-Location Newsvendor

2(a) Expected profit without transshipment

Without transshipment each store can only sell its own stock, so the network is the dedicated design of Question 1 with store inventories as capacities: sales at store i equal min{D_i, s_i}. The procurement cost of 50 x 1,300 = $65,000 is incurred up front and unsold units are worthless, so:

$$\mathbb{E}[\text{profit}] = 100 \sum_{i=1}^{3} \mathbb{E}[\min\{D_i,\ s_i\}] - 50 \sum_{i=1}^{3} s_i, \qquad s = (300,\ 500,\ 500).$$

Evaluating the expectation with 10,000 demand samples gives expected sales of 1,268.0 units and an expected profit of about $61,799 (standard error about $28).

Answer: The total expected profit is approximately $61,799.

The figure can be checked in closed form. Each store stocks exactly its mean demand, and the truncation at zero is negligible because every mean is at least 12.5 standard deviations above zero. For a normal demand stocked at its mean, the expected shortfall is sigma times the standard normal density at zero, about 0.399 sigma, so the expected lost sales are 0.399 x (20 + 20 + 40) = 31.9 units and the expected profit is $61,808, in line with the simulation.

2(b) Expected profit without demand uncertainty

If sigma_i = 0 every store sells exactly its mean demand, which equals its stock, so all 1,300 units are sold at full price.

Answer: The profit is 100 x 1,300 - 50 x 1,300 = $65,000.

The gap of about $3,201 between (b) and (a) is the cost of demand uncertainty under the current allocation: stores with excess demand lose sales, while stores with weak demand are left with stock nobody can buy. Harbour Front, with the largest standard deviation, contributes half of it.

2(c) Should the retailer adopt transshipment?

Once demand is realised, transshipment turns the dedicated network into a complete one: a unit of store i's inventory can be sold at store j for a margin of 100 - c_ij, where c_ii = 0. The best use of the realised inventory is again a network flow problem, in which x_ij is the number of units of store i's stock sold to customers at store j:

$$f(s, D) = \max \quad \sum_{i=1}^{3} \sum_{j=1}^{3} (100 - c_{ij})\, x_{ij}$$

$$\text{s.t.} \quad \sum_{j=1}^{3} x_{ij} \le s_i, \qquad \sum_{i=1}^{3} x_{ij} \le D_j, \qquad x_{ij} \ge 0.$$

For the example in the question, with demand (250, 550, 450), Orchard is 50 units short while Jurong West and Harbour Front each have 50 units left. The cheapest source is Harbour Front at $7 per unit, so 50 units are shipped and the retailer gains 50 x (100 - 7) - 200 = $4,450 compared with not using the service. On average, the retailer should adopt the service if the expected increase in the optimal second-stage revenue exceeds the fixed fee of $200. The expected increase is estimated by solving both networks on the same demand samples:

Samples	Profit without	Profit with (after $200 fee)	Gross gain	Net gain	Std. error
100	$61,933	$62,982	$1,249	$1,049	$113
500	$61,547	$62,482	$1,135	$935	$51
1,000	$61,839	$62,677	$1,038	$838	$33
5,000	$61,803	$62,658	$1,055	$855	$16
10,000	$61,811	$62,690	$1,078	$878	$11

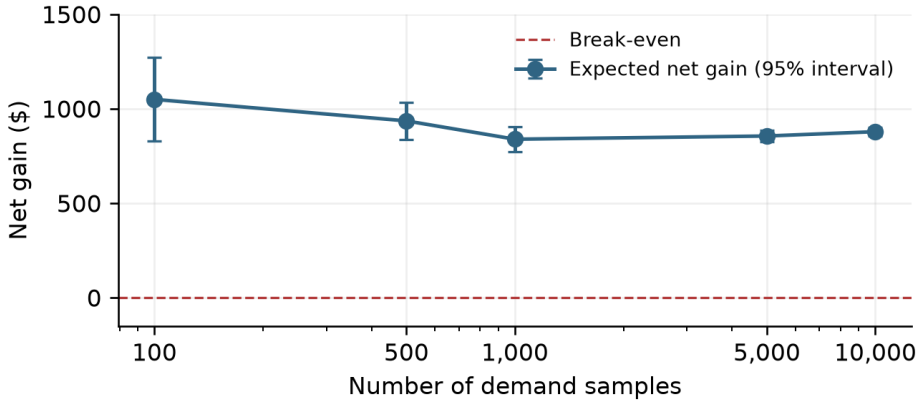


Figure 2. Expected net gain from transshipment by sample size

Answer: Yes. Transshipment raises the expected revenue by about $1,078, so after the $200 fee the expected profit rises by about $878 to roughly $62,690.

The estimate is noisy for small samples, but every sample size gives a net gain far above zero, and from 1,000 samples onwards the estimate settles between about $840 and $880 with a standard error of $11 to $33. The recommendation is therefore robust to the sample size. The gross gain recovers roughly a third of the cost of uncertainty identified in (b); the rest is lost on samples in which every store is short or every store has surplus, where moving stock between stores cannot help.

2(d) Two-stage stochastic optimization of the inventory positions

The inventory positions s = (s1, s2, s3) are chosen in the first stage, before demand is known, at a cost of $50 per unit. In the second stage the demand D is realised and the retailer sells and transships optimally, earning f(s, D) from part (c). The fixed fee is sunk and can be left out. The two-stage problem is:

$$\max_{s \geq 0} \quad -50 \sum_{i=1}^{3} s_i + \mathbb{E}[f(s, D)],$$

$$f(s, D) = \max_{x \geq 0} \left\{ \sum_{i,j} (100 - c_{ij})\, x_{ij} \;:\; \sum_{j} x_{ij} \leq s_i \;\forall i, \quad \sum_{i} x_{ij} \leq D_j \;\forall j \right\}.$$

Replacing the expectation with the average over K demand samples D^1, ..., D^K, and giving every sample its own copy of the second-stage decisions, yields the sample average approximation, which is a single linear program:

$$\max_{s,\,x} \quad -50\sum_{i=1}^{3} s_i + \frac{1}{K}\sum_{k=1}^{K}\sum_{i=1}^{3}\sum_{j=1}^{3}(100 - c_{ij})\,x_{ijk}$$

$$\text{s.t.} \quad \sum_{j=1}^{3} x_{ijk} \le s_i, \qquad i = 1,2,3;\ k = 1,\ldots,K,$$

$$\sum_{i=1}^{3} x_{ijk} \le D_j^k, \qquad j = 1,2,3;\ k = 1,\ldots,K,$$

$$s_i \ge 0, \qquad x_{ijk} \ge 0.$$

The model was solved with Gurobi for several sample sizes, and each solution was then evaluated on a separate set of 10,000 test samples to see how it performs out of sample:

Samples K	Jurong West	Orchard	Harbour Front	Total	Out-of-sample profit
50	301.4	495.0	498.4	1294.7	$62,880
100	292.5	500.2	500.5	1293.2	$62,869
200	298.6	501.2	499.4	1299.3	$62,893
Current (300, 500, 500)	300	500	500	1,300	$62,893

Answer: The SAA solutions stay within a few units of (300, 500, 500), and on the test samples none of them beats the current allocation. Matching each store's inventory to its mean demand is therefore close to optimal even with transshipment.

The result has a simple explanation. With a price of $100, a cost of $50 and no salvage value, the cost of one unit short equals the cost of one unit left over, so the best single-store stock is the median of demand, which is the mean for a normal distribution. Transshipment adds recourse after demand is known, but each store's demand is still symmetric around its mean and the shipping costs are small next to the $50 margin, so the balance between shortage and surplus barely moves. The value of transshipment comes from rebalancing stock after demand is realised, not from changing how much is stocked before the season.

3. Transportation Problem

3(a) Linear programming formulation

Let x_ij be the amount shipped from supply node i to demand node j. Total supply a + (100 - a) = 100 equals total demand 70 + 30, so the problem is balanced and every constraint is an equality:

$$\min \quad c_{11}x_{11} + c_{12}x_{12} + c_{21}x_{21} + c_{22}x_{22}$$

$$\text{s.t.} \quad x_{11} + x_{12} = a, \qquad x_{21} + x_{22} = 100 - a,$$

$$x_{11} + x_{21} = 70, \qquad x_{12} + x_{22} = 30, \qquad x_{ij} \geq 0.$$

With x = (x11, x12, x21, x22) and the constraints written as Ax = b, the constraint matrix is:

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

Rows: supply 1, supply 2, demand 1, demand 2; b = (a, 100 - a, 70, 30)

Each column has exactly two nonzero entries, one in a supply row and one in a demand row, which is the network structure of a transportation problem. The two supply rows and the two demand rows add up to the same vector, so one constraint is redundant and the rank of A is 3 = m + n - 1. A basic solution therefore has three basic arcs, which form a spanning tree of the four nodes.

3(b) Flow on the spanning tree without arc (1, 1)

Deleting arc (1, 1) leaves the three arcs (1, 2), (2, 1) and (2, 2). They connect all four nodes without a cycle, so they form a spanning tree and determine a unique solution of Ax = b. The flows follow by working in from the leaves of the tree:

$$\text{Demand 1 is served only by supply 2}: \quad x_{21} = 70,$$

$$\text{Supply 1 ships only to demand 2}: \quad x_{12} = a,$$

$$\text{Demand 2 balance}: \quad x_{22} = 30 - x_{12} = 30 - a.$$

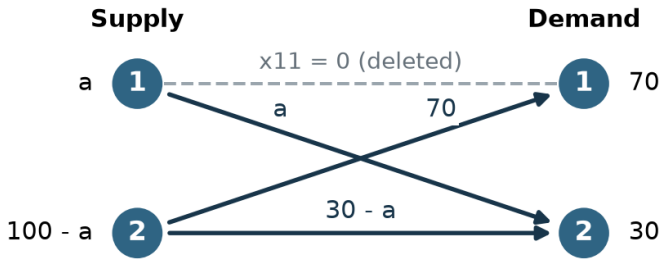


Figure 3. Spanning-tree flow after deleting arc (1, 1)

Answer: x11 = 0, x12 = a, x21 = 70, x22 = 30 - a. The remaining equation, supply 2, is satisfied automatically: 70 + (30 - a) = 100 - a.

3(c) Range of supply for feasibility

The tree solution already satisfies every equality, so it is feasible exactly when all four flows are nonnegative. The flows 0 and 70 always are; the other two require a >= 0 and 30 - a >= 0.

Answer: The solution is feasible, and is then a basic feasible solution (an extreme point), exactly when 0 <= a <= 30.

At the end points a = 0 and a = 30 one of the basic flows is zero, so the basic feasible solution is degenerate. For a > 30, supply node 1 holds more than demand node 2 can absorb, so part of its supply must go to demand node 1 and a feasible basis has to include arc (1, 1).

4. Logical Constraints with Binary Variables

Each statement is an implication between binary variables. A linear inequality models it correctly when it rules out exactly the assignments that violate the statement and leaves all other assignments feasible.

4(a) z = 1 if at least one x_i = 1

$$z \geq x_i, \qquad i = 1, \ldots, n.$$

If some x_i = 1, the constraint for that index forces z = 1. If every x_i = 0, the constraints read z >= 0 and leave z free. An equivalent single constraint is n z >= x_1 + ... + x_n.

4(b) If all x_i = 0, then z = 0

$$z \leq \sum_{i=1}^{n} x_i.$$

If every x_i = 0, the right-hand side is zero and z = 0. If at least one x_i = 1, the right-hand side is at least one and z is free. Together with (a), this makes z the logical OR of the x_i.

4(c) z = 1 if all x_i = 1

$$z \geq \sum_{i=1}^{n} x_i - (n-1).$$

If every x_i = 1, the right-hand side equals 1 and forces z = 1. If any x_i = 0, the right-hand side is at most 0 and z is free.

4(d) If any x_i = 0, then z = 0

$$z \leq x_i, \qquad i = 1, \ldots, n.$$

If some x_i = 0, the constraint for that index forces z = 0. If every x_i = 1, the constraints read z <= 1 and leave z free. Together with (c), this makes z the product x_1 x_2 ... x_n, the n-variable version of linearising a product of binary variables.

4(e) If x_1 + ... + x_n >= 5, then z = 1

$$\sum_{i=1}^{n} x_i - 4 \leq (n-4)\, z.$$

It is easiest to read the contrapositive: if z = 0, then at most four x_i can equal 1. With z = 0 the constraint becomes x_1 + ... + x_n <= 4, which is exactly that. With z = 1 it becomes x_1 + ... + x_n <= n, which always holds, so z = 1 never cuts off a valid assignment. The coefficient n - 4 is the smallest big-M that works, since the sum can exceed 4 by at most n - 4 (for n < 5 the premise can never hold and no constraint is needed).

5. Economic Lot-Sizing

5(a) Integer programming formulation

Let x_t be the inventory at the end of season t, y_t the number of airplanes produced in season t, and z_t = 1 if the production facilities are set up in season t. With demand d_t, setup cost K_t and holding cost h_t, all in millions of dollars:

$$\min \quad \sum_{t=1}^{4} (K_t z_t + h_t x_t)$$

$$\text{s.t.} \quad x_{t-1} + y_t - x_t = d_t, \qquad t = 1, \ldots, 4, \qquad x_0 = 0,$$

$$y_t \le M z_t, \qquad t = 1, \ldots, 4, \qquad M = \sum_{t=1}^{4} d_t = 10,$$

$$x_t,\ y_t \ge 0 \text{ and integer}, \qquad z_t \in \{0, 1\}.$$

The first set of constraints is the inventory balance: stock carried in plus production equals deliveries plus stock carried out. The big-M constraints link production to setups. If z_t = 0 nothing can be produced, and if z_t = 1 production is effectively unrestricted, since it never pays to make more than the 10 airplanes ordered. Requiring x_t >= 0 means every delivery is met on time, and the holding cost is charged on the inventory left at the end of each season, as in the worked example of Table 3.4 (which this model prices at 7 million).

5(b) Optimal production schedule

Gurobi finds a minimum total cost of 4.8 million dollars. The optimum is not unique: the solver's solution pool contains two schedules with the same cost.

Schedule	Production (y1 to y4)	End inventory (x1 to x4)	Setup cost	Holding cost	Total
Optimal 1	10, 0, 0, 0	7, 5, 2, 0	2.0	2.8	4.8
Optimal 2	5, 0, 5, 0	2, 0, 2, 0	4.0	0.8	4.8
Table 3.4 example	3, 6, 0, 1	0, 4, 1, 0	6.0	1.0	7.0

Answer: The minimum cost is $4.8 million. One optimal schedule is to produce all 10 airplanes in season 1; the other is to produce 5 in season 1 and 5 in season 3. Both save $2.2 million compared with the schedule in Table 3.4.

The tie comes from the cost ratio. A setup costs 2 million, the same as holding ten airplanes for one season. Moving the season 3 and season 4 airplanes into the season 1 batch saves one setup and adds holding of 3 x 0.2 x 2 + 2 x 0.2 x 2 = 2.0 million, so the two schedules cost the same. Both schedules also produce only when the inventory carried in is zero: production in a season covers whole seasons of demand, never a fraction of one.

As an independent check, the table below lists every setup pattern. A setup in season 1 is required because three airplanes are due then, and given the setup seasons it is optimal to produce, in each setup season, exactly the demand up to the next setup. The two optimal patterns are the ones Gurobi found.

Setup in seasons (z1 z2 z3 z4)	Production	Total cost (million)
1 0 0 0	10, 0, 0, 0	4.8
1 0 1 0	5, 0, 5, 0	4.8
1 1 0 0	3, 7, 0, 0	5.4
1 0 0 1	8, 0, 0, 2	5.6
1 0 1 1	5, 0, 3, 2	6.4
1 1 1 0	3, 2, 5, 0	6.4
1 1 0 1	3, 5, 0, 2	6.6
1 1 1 1	3, 2, 3, 2	8.0

6. TSP with Time Windows

6(a) Minimum travel time without waiting-time limits

Let x_ij = 1 if the delivery man walks directly from location i to location j, and let u_i be the time at which he reaches location i, with u_1 = 0 at the restaurant. The model is a travelling salesman problem with assignment constraints and time-accumulating subtour-breaking constraints:

$$\min \quad \sum_{i=1}^{6}\sum_{j=1}^{6} c_{ij}x_{ij}$$

$$\text{s.t.} \quad \sum_{j=1}^{6} x_{ij} = 1 \;\; (i = 1, \ldots, 6), \qquad \sum_{i=1}^{6} x_{ij} = 1 \;\; (j = 1, \ldots, 6),$$

$$u_i + c_{ij} - u_j \le M(1 - x_{ij}), \qquad i \ne j, \;\; j = 2, \ldots, 6, \qquad u_1 = 0,$$

$$x_{ij} \in \{0, 1\}, \qquad u_i \ge 0.$$

If x_ij = 1 the third constraint reads u_j >= u_i + c_ij, so u_j is the arrival time at j; if x_ij = 0 the large constant M (1,000 here) makes it inactive. Because arrival times must increase along every arc that does not return to location 1, no subtour can avoid location 1, and every feasible solution is a single tour. A cost of 1,000 on the diagonal of the time matrix rules out staying in place.

Answer: The minimum travelling time is 22 minutes. Gurobi returns the route 1 -> 2 -> 4 -> 6 -> 5 -> 3 -> 1.

The route is not unique; enumerating all 120 tours gives 12 routes of 22 minutes. The locations form two clusters: the restaurant with customers 2 and 3, close together near VivoCity, and customers 4, 5 and 6 about 6 to 7 minutes away, which are 1 or 2 minutes from one another. Every tour must leave the first cluster through customer 2 or 3 and come back through the other, so it has the form 1, 2 or 3, the far cluster, 3 or 2, 1. The two legs from the restaurant cost 3 + 3, and the best combinations of crossings and paths through the far cluster cost 16 minutes, for example 7 + 2 + 7 for 2, 4, 5, 6, 3, or 6 + 3 + 7 for 2, 5, 4, 6, 3. Several orders achieve this, which explains the ties.

6(b) Minimum travel time with waiting-time limits

Because u_j measures the arrival time, the delivery commitment is a simple upper bound on each customer's arrival time, W_j, added to the model in (a):

$$u_j \le W_j, \qquad W = (W_2, \ldots, W_6) = (5,\ 10,\ 15,\ 13,\ 14) \text{ minutes.}$$

Location	Arrival time	Maximum waiting time	Slack
2	3 min	5 min	2 min
3	6 min	10 min	4 min
5	12 min	13 min	1 min
6	13 min	14 min	1 min
4	15 min	15 min	0 min
Back at 1	25 min	-	-

Answer: Yes, a feasible route exists. The fastest one is 1 -> 2 -> 3 -> 5 -> 6 -> 4 -> 1, which takes 25 minutes and meets every customer's waiting-time limit.

The route is forced almost step by step. Customer 2 must be reached within 5 minutes, which is only possible by going there first (arrival at 3), and customer 3 must then be reached by minute 10, so he comes second (arrival at 6). Of the six orders through customers 4, 5 and 6 starting at minute 6, only 5, 6, 4 meets every limit: starting with 4 or 6 reaches customer 5 at minute 14 or later, past its limit of 13, and the order 5, 4, 6 reaches customer 6 at minute 15, past its limit of 14. Enumerating all 120 tours confirms that this route is the unique feasible optimum. The service commitment costs 3 minutes of total travel time compared with (a), because the walk back to the restaurant now starts from customer 4 (10 minutes) rather than from customer 3 (3 minutes).

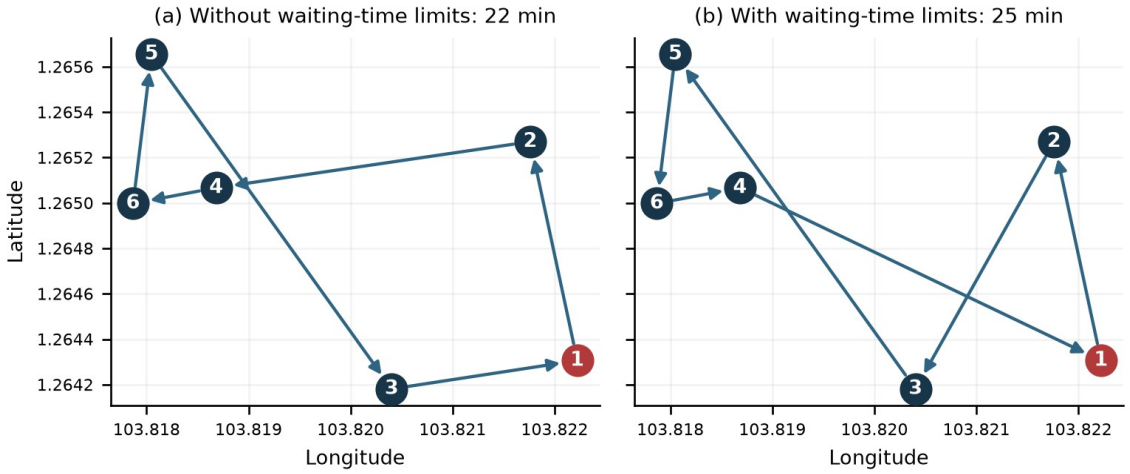


Figure 4. Optimal delivery routes, plotted by latitude and longitude (location 1 in red)

Appendix: Computational Code

The following Gurobi code produced the numerical results reported for Questions 1, 2, 5 and 6.

Question 1: Process flexibility

```
from gurobipy import *
import numpy as np

#########Parameters Set-up############

N = 6                               # number of plants and of models
supply = np.full(N, 100)            # weekly capacity at each plant
mean = np.full(N, 100)              # mean demand of each model
cov = np.diag(np.full(N, 30**2))    # independent demands with s.d. 30

DESIGNS = {
    "Dedicated":     tuplelist([(i, i) for i in range(N)]),
    "Open chain":    tuplelist([(i, i) for i in range(N)] + [(i, i + 1) for i in range(N - 1)]),
    "Long chain":    tuplelist([(i, i) for i in range(N)] + [(i, (i + 1) % N) for i in range(N)]),
    "Full flexible": tuplelist([(i, j) for i in range(N) for j in range(N)]),
}


#########Model Set-up Using Function############
def model_setup(ARCS):
    m = Model("Process_Flexi")
    m.setParam("OutputFlag", 0)

    # x[i,j]: amount of model j produced at plant i
    x = m.addVars(ARCS, name="x")

    # maximize total sales
    m.setObjective(quicksum(x[i, j] for (i, j) in ARCS), GRB.MAXIMIZE)

    # capacity constraint at each plant
    m.addConstrs((quicksum(x[i, j] for (i, j) in ARCS.select(i, "*")) <= supply[i] for i in range(N)),
"capacity")

    # demand constraint for each model (right-hand side is reset for every sample)
    demand_con = m.addConstrs((quicksum(x[i, j] for (i, j) in ARCS.select("*", j)) <= mean[j] for j in
range(N)), "demand")
    return m, demand_con


def evaluate(ARCS, demand_samples):
    m, demand_con = model_setup(ARCS)
    sales = np.zeros(len(demand_samples))
    for k, demand in enumerate(demand_samples):
        for j in range(N):
            demand_con[j].RHS = demand[j]
        m.optimize()
        sales[k] = m.ObjVal
    return sales


def solve(sample_size=1000, seed=5103):
    np.random.seed(seed)
    # demand is sampled from the multivariate normal distribution and truncated above zero;
    # the same samples are used for every design so the designs are compared fairly
    demand_samples = np.maximum(np.random.multivariate_normal(mean, cov, sample_size), 0)
    return {name: evaluate(ARCS, demand_samples) for name, ARCS in DESIGNS.items()}


if __name__ == "__main__":
    for size in (1000, 10000):
        sales = solve(size)
        print("\nSample size", size)
        for name, v in sales.items():
            print("%-14s average sales %.2f (std. error %.2f)" % (name, v.mean(), v.std(ddof=1) /
np.sqrt(size)))
```

Question 2: Multi-location newsvendor and transshipment

```
from gurobipy import *
import numpy as np

#########Parameters Set-up############

STORES = ["Jurong West", "Orchard", "Harbour Front"]
N = len(STORES)
price, unit_cost = 100, 50
mu = np.array([300, 500, 500])
sigma = np.array([20, 20, 40])
inventory = np.array([300, 500, 500])
fixed_fee = 200

# unit transshipment cost from store i to store j
ship = np.array([[0, 22, 19],
                 [22, 0, 7],
                 [19, 7, 0]])

DEDICATED = tuplelist([(i, i) for i in range(N)])
TRANSSHIP = tuplelist([(i, j) for i in range(N) for j in range(N)])


def sample_demand(K, seed, sd=sigma):
    # independent normal demands truncated above zero
    rng = np.random.default_rng(seed)
    return np.maximum(rng.normal(mu, sd, size=(K, N)), 0)


#########Second-stage Model: Sales after Demand Is Realized############
def model_setup(ARCS):
    m = Model("multi_location")
    m.setParam("OutputFlag", 0)

    # x[i,j]: units of store i's inventory sold to customers at store j
    x = m.addVars(ARCS, name="x")

    # sales revenue net of transshipment cost
    m.setObjective(quicksum((price - ship[i, j]) * x[i, j] for (i, j) in ARCS), GRB.MAXIMIZE)

    # inventory constraint at each store (right-hand sides are reset later)
    inv_con = m.addConstrs((quicksum(x[i, j] for (i, j) in ARCS.select(i, "*")) <= inventory[i] for i in
range(N)), "inventory")
    # demand constraint at each store
    dem_con = m.addConstrs((quicksum(x[i, j] for (i, j) in ARCS.select("*", j)) <= mu[j] for j in range(N)),
"demand")
    return m, inv_con, dem_con


def evaluate(ARCS, stock, demand_samples):
    # average second-stage revenue over the demand samples for a given inventory position
    m, inv_con, dem_con = model_setup(ARCS)
    for i in range(N):
        inv_con[i].RHS = stock[i]
    value = np.zeros(len(demand_samples))
    for k, demand in enumerate(demand_samples):
        for j in range(N):
            dem_con[j].RHS = demand[j]
        m.optimize()
        value[k] = m.ObjVal
    return value


#########Sample Average Approximation for the Inventory Positions############
def solve_saa(demand_samples):
    K = len(demand_samples)
    m = Model("SAA")
    m.setParam("OutputFlag", 0)

    # first stage: inventory position at each store
    s = m.addVars(N, name="s")
    # second stage: x[i,j,k] units of store i's stock sold at store j in sample k
    x = m.addVars(N, N, K, name="x")

    m.setObjective(-unit_cost * quicksum(s[i] for i in range(N))
                   + (1.0 / K) * quicksum((price - ship[i, j]) * x[i, j, k]
                                          for i in range(N) for j in range(N) for k in range(K)),
                   GRB.MAXIMIZE)
```

```
    m.addConstrs((quicksum(x[i, j, k] for j in range(N)) <= s[i] for i in range(N) for k in range(K)),
"inventory")
    m.addConstrs((quicksum(x[i, j, k] for i in range(N)) <= demand_samples[k, j] for j in range(N) for k in
range(K)), "demand")
    m.optimize()
    return np.array([s[i].X for i in range(N)]), m.ObjVal


def solve():
    res = {}
    procurement = unit_cost * inventory.sum()

    # (a) expected profit without transshipment
    D = sample_demand(10000, seed=1)
    sales = evaluate(DEDICATED, inventory, D) / price
    res["a"] = {"profit": price * sales.mean() - procurement, "sales": sales.mean(),
                "se": price * sales.std(ddof=1) / np.sqrt(len(D))}

    # (b) no demand uncertainty: demand equals its mean
    sales_det = evaluate(DEDICATED, inventory, mu.reshape(1, -1))
    res["b"] = {"profit": sales_det[0] - procurement}

    # (c) value of transshipment for different sample sizes
    res["c"] = []
    for K in (100, 500, 1000, 5000, 10000):
        D = sample_demand(K, seed=K)
        base = evaluate(DEDICATED, inventory, D)
        trans = evaluate(TRANSSHIP, inventory, D)
        gain = trans - base
        res["c"].append({"K": K, "profit_without": base.mean() - procurement,
                         "profit_with": trans.mean() - procurement - fixed_fee,
                         "gain": gain.mean(), "net": gain.mean() - fixed_fee,
                         "se": gain.std(ddof=1) / np.sqrt(K)})

    # (d) optimal inventory positions under transshipment by SAA
    res["d"] = []
    D_test = sample_demand(10000, seed=2026)
    for K in (50, 100, 200):
        stock, saa_value = solve_saa(sample_demand(K, seed=100 + K))
        test = evaluate(TRANSSHIP, stock, D_test).mean() - unit_cost * stock.sum()
        res["d"].append({"K": K, "stock": stock, "saa": saa_value, "test": test})
    res["d_benchmark"] = evaluate(TRANSSHIP, inventory, D_test).mean() - procurement
    return res


if __name__ == "__main__":
    res = solve()
    print("(a) expected profit: %.2f (expected sales %.2f, s.e. %.2f)" % (res["a"]["profit"], res["a"]
["sales"], res["a"]["se"]))
    print("(b) deterministic profit: %.2f" % res["b"]["profit"])
    for r in res["c"]:
        print("(c) K=%5d  gross gain %.2f  net gain %.2f  (s.e. %.2f)" % (r["K"], r["gain"], r["net"],
r["se"]))
    for r in res["d"]:
        print("(d) K=%3d  s = %s  SAA value %.2f  out-of-sample profit %.2f"
              % (r["K"], np.round(r["stock"], 2), r["saa"], r["test"]))
    print("(d) out-of-sample profit of (300, 500, 500): %.2f" % res["d_benchmark"])
```

Question 5: Economic lot-sizing

```
from gurobipy import *
import numpy as np

#########Parameters Set-up#############

d = np.array([3, 2, 3, 2])          # demand in each season
K = np.array([2, 2, 2, 2])          # setup cost (million $)
h = np.array([0.2, 0.2, 0.2, 0.2])  # holding cost per airplane per season (million $)
T = len(d)
M = d.sum()                         # big M: never produce more than the total demand


#########Model Set-up#############
def model_setup():
    m = Model("lot_sizing")
```

```
    m.setParam("OutputFlag", 0)

    x = m.addVars(T, vtype=GRB.INTEGER, name="x")   # inventory at the end of season t
    y = m.addVars(T, vtype=GRB.INTEGER, name="y")   # airplanes produced in season t
    z = m.addVars(T, vtype=GRB.BINARY, name="z")    # 1 if production is set up in season t

    # setup cost plus holding cost
    m.setObjective(quicksum(K[t] * z[t] + h[t] * x[t] for t in range(T)), GRB.MINIMIZE)

    # inventory balance with zero initial inventory
    m.addConstr(y[0] - x[0] == d[0], "balance[0]")
    m.addConstrs((x[t - 1] + y[t] - x[t] == d[t] for t in range(1, T)), "balance")

    # production is possible only if the facilities are set up
    m.addConstrs((y[t] <= M * z[t] for t in range(T)), "setup")
    return m, x, y, z


def solve():
    m, x, y, z = model_setup()
    # keep every optimal solution in the solution pool to detect ties
    m.setParam("PoolSearchMode", 2)
    m.setParam("PoolSolutions", 50)
    m.setParam("PoolGap", 0)
    m.optimize()

    optimal = []
    for k in range(m.SolCount):
        m.setParam("SolutionNumber", k)
        sol = {"cost": m.PoolObjVal,
               "y": [round(y[t].Xn) for t in range(T)],
               "x": [round(x[t].Xn) for t in range(T)],
               "z": [round(z[t].Xn) for t in range(T)]}
        if abs(sol["cost"] - m.ObjVal) < 1e-6 and all(sol["y"] != s["y"] for s in optimal):
            optimal.append(sol)
    return optimal[0], optimal


if __name__ == "__main__":
    best, alternatives = solve()
    print("Minimum total cost: %.2f million" % best["cost"])
    for sol in alternatives:
        print("production", sol["y"], "inventory", sol["x"], "setup", sol["z"])
```

Question 6: TSP with time windows

```
from gurobipy import *
import numpy as np

#########Parameters Set-up############

# pairwise walking time (minutes) between locations 1-6; 1000 on the diagonal forbids self-loops
cost = np.array([[1000,    3,    3,   10,    9,   10],
                 [   3, 1000,    3,    7,    6,    7],
                 [   3,    3, 1000,    7,    6,    7],
                 [  10,    7,    7, 1000,    1,    2],
                 [   9,    6,    6,    1, 1000,    1],
                 [  10,    7,    7,    2,    1, 1000]])

# maximum waiting time of the customers at locations 2-6 (location 1 is the restaurant)
max_wait = np.array([0, 5, 10, 15, 13, 14])

N = cost.shape[0]
M = 1000   # big M


#########Model Set-up############
def model_setup(time_window):
    tsp = Model("tsp_time_window")
    tsp.setParam("OutputFlag", 0)

    x = tsp.addVars(N, N, vtype=GRB.BINARY, name="x")   # 1 if the route goes from i to j
    u = tsp.addVars(N, name="u")                         # arrival time at node i

    tsp.setObjective(quicksum(cost[i, j] * x[i, j] for i in range(N) for j in range(N)), GRB.MINIMIZE)
```

```
    # assignment constraints: leave and enter every location exactly once
    tsp.addConstrs((quicksum(x[i, j] for j in range(N)) == 1 for i in range(N)), "out")
    tsp.addConstrs((quicksum(x[i, j] for i in range(N)) == 1 for j in range(N)), "in")

    # accumulated walking time; it also breaks every subtour that avoids location 1
    tsp.addConstr(u[0] == 0, "start")
    tsp.addConstrs((u[i] + cost[i, j] - u[j] <= M * (1 - x[i, j])
                    for i in range(N) for j in range(1, N) if i != j), "time")

    # deliver before the maximum waiting time of each customer
    if time_window:
        tsp.addConstrs((u[j] <= max_wait[j] for j in range(1, N)), "window")
    return tsp, x, u


def route_from(x):
    # follow the arcs with x[i,j] = 1 starting from location 1
    succ = {i: j for i in range(N) for j in range(N) if x[i, j].X > 0.5}
    route, node = [1], 0
    while True:
        node = succ[node]
        route.append(node + 1)
        if node == 0:
            return route


def arrival_times(route):
    t, times = 0, {}
    for a, b in zip(route, route[1:]):
        t += cost[a - 1, b - 1]
        times[b] = t
    return times


def solve(time_window):
    tsp, x, u = model_setup(time_window)
    tsp.optimize()
    if tsp.Status != GRB.OPTIMAL:
        return None
    route = route_from(x)
    return {"time": tsp.ObjVal, "route": route, "arrival": arrival_times(route)}


if __name__ == "__main__":
    for tw in (False, True):
        res = solve(tw)
        print("\nWith time windows:" if tw else "\nWithout time windows:")
        print("Route:", " -> ".join(map(str, res["route"])), " total time:", res["time"])
        print("Arrival times:", res["arrival"])
```

AI Declaration:
I used Claude Opus, OpenAI GPT5.6-Sol to generate ideas and formatting. I am responsible for the content and quality of the submitted work